

Name: Krish Singh

Roll: AID25072

B.Tech Artificial Intelligence & Data Science - 2nd Semester

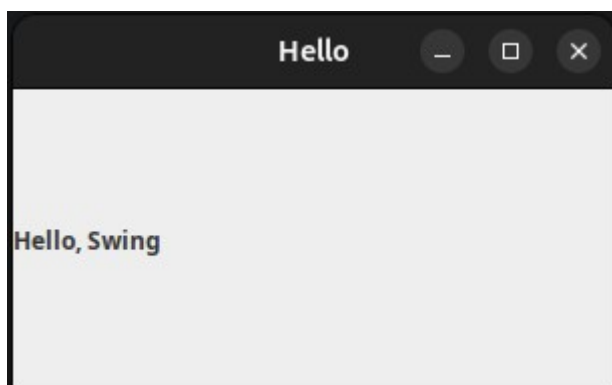
Object Oriented Programming – Lab Assignment 11

LAB Sheet Solutions:

HelloGUI:

```
import javax.swing.*;
public class HelloGUI{
    public static void main(String[] args){
        JFrame frame = new JFrame("Hello");
        JLabel label = new JLabel("Hello, Swing");
        frame.add(label);
        frame.setSize(300,150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Output:



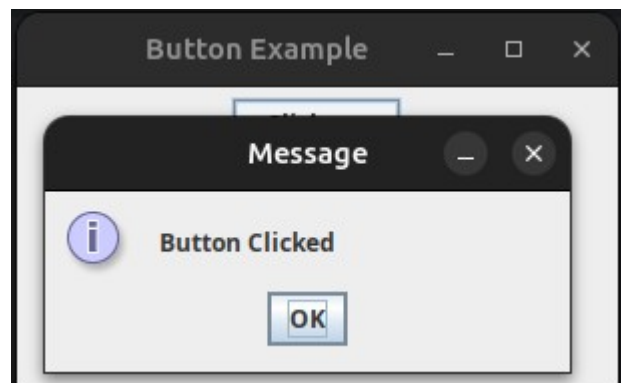
ButtonClick.java:

Code:

```
import javax.swing.*;
import java.awt.*;
public class ButtonClick{
    public static void main(String[] args){
        JFrame frame = new JFrame("Button Example");
        frame.setLayout(new FlowLayout());
        JButton button = new JButton("Click me");

        button.addActionListener(e->{
            JOptionPane.showMessageDialog(frame,"Button Clicked");
        });
        frame.add(button);
        frame.setSize(300,150);
        frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

Output:

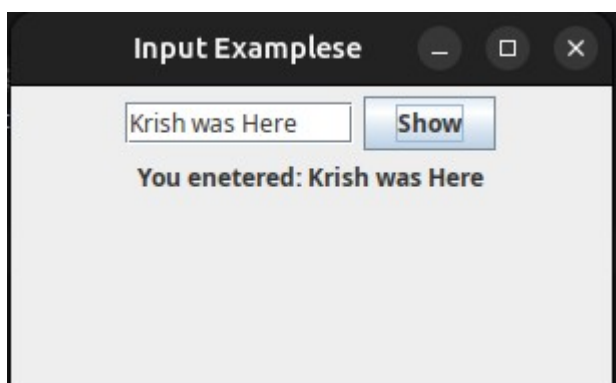
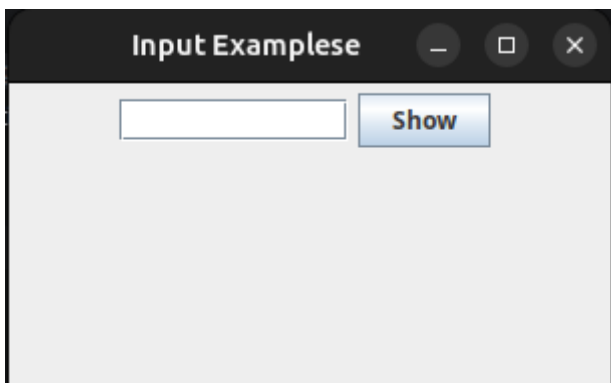


InputDisplay.java:

Code:

```
import javax.swing.*;
import java.awt.*;
public class InputDisplay {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Input Examplese");
        frame.setLayout(new FlowLayout(FlowLayout.CENTER));
        JTextField input = new JTextField(10);
        JButton button = new JButton("Show");
        JLabel label = new JLabel();
        button.addActionListener(e->{
            label.setText("You enetered: "+input.getText());
        });
        frame.add(input);
        frame.add(button);
        frame.add(label);
        frame.setSize(300,150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Output:

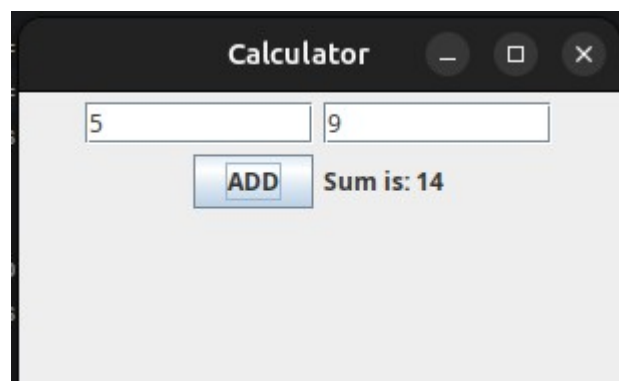
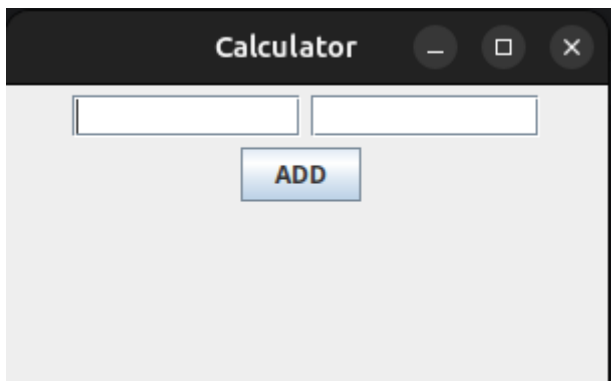


Calculator.java:

Code:

```
import javax.swing.*;
import java.awt.*;
public class Calculator{
    public static void main(String[] args){
        JFrame frame = new JFrame("Calculator");
        frame.setLayout(new FlowLayout());
        JTextField num1 = new JTextField(10);
        JTextField num2 = new JTextField(10);
        JButton add = new JButton("ADD");
        JLabel label = new JLabel();
        add.addActionListener(e->{
            try{
                int a = Integer.parseInt(num1.getText());
                int b = Integer.parseInt(num2.getText());
                label.setText("Sum is: "+(a+b));
            }
            catch(Exception ex){
                label.setText("Invalid Input");
            }
        });
        frame.add(num1);
        frame.add(num2);
        frame.add(add);
        frame.add(label);
        frame.setSize(300,150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Output:



ToDoApp.java:

Code:

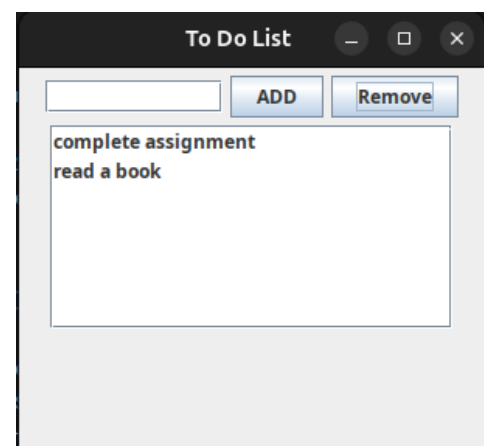
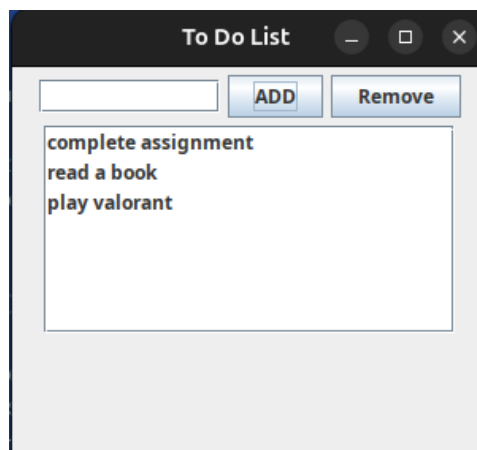
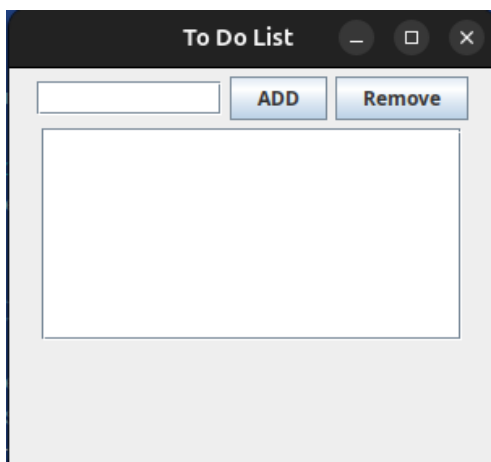
```
import java.awt.*;
import javax.swing.*;
public class ToDoApp{
    public static void main(String[] args){
        JFrame frame = new JFrame("To Do List");
        frame.setLayout(new FlowLayout());

        JTextField input = new JTextField(10);
        JButton add = new JButton("ADD");
        JButton remove = new JButton("Remove");

        DefaultListModel<String> model = new DefaultListModel<>();
        JList<String> list = new JList<>(model);

        add.addActionListener(e->{
            if(!input.getText().isEmpty()){
                model.addElement(input.getText());
                input.setText("");
            }
        });
        remove.addActionListener(e->{
            int index = list.getSelectedIndex();
            if(index!=-1){
                model.remove(index);
            }
        });
        frame.add(input);
        frame.add(add);
        frame.add(remove);
        frame.add(new JScrollPane(list));
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(300,250);
        frame.setVisible(true);
    }
}
```

Output:



FoodOrderApp.java:

Code:

```
import javax.swing.*.*;
import java.awt.*.*;

public class FoodOrderApp extends JFrame {
    private JCheckBox pizza, burger, juice;
    private JTextArea output;
    public FoodOrderApp() {
        setTitle("Food Ordering app");
        setSize(500,500);
        setLayout(new FlowLayout());
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        pizza = new JCheckBox(" Pizza -> 100 rs");
        burger = new JCheckBox("Burger -> 80 rs");
        juice = new JCheckBox("juice -> 50rs");

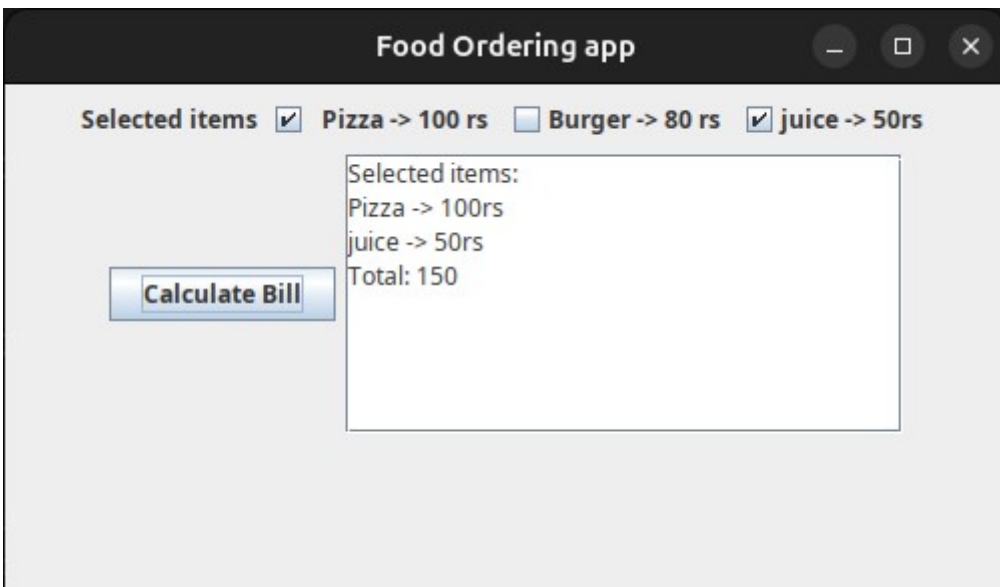
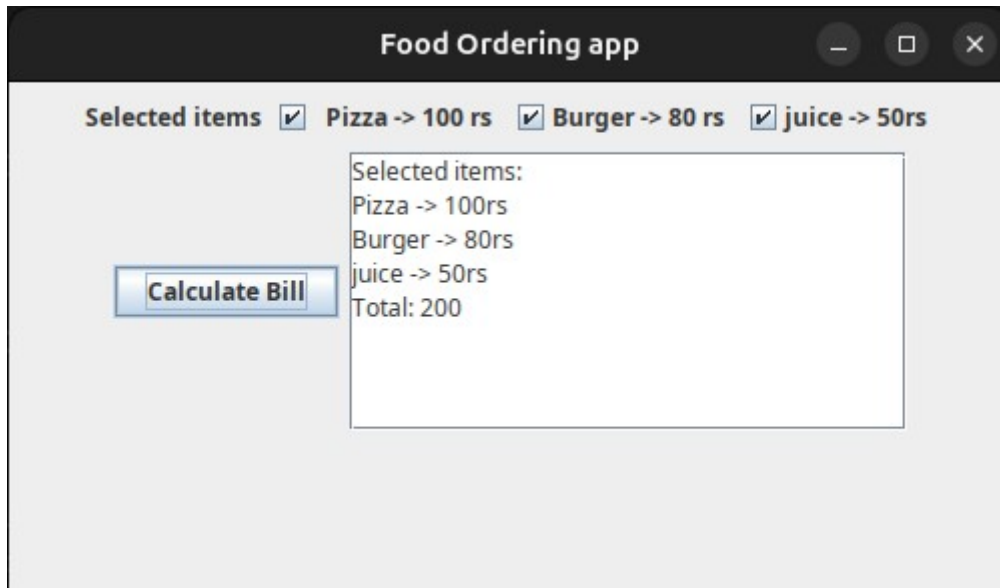
        JButton billButton = new JButton("Calculate Bill");
        output = new JTextArea(8,25);
        output.setEditable(false);

        add(new JLabel("Selected items"));
        add(pizza);
        add(burger);
        add(juice);
        add(billButton);
        add(new JScrollPane(output));
        setVisible(true);

        billButton.addActionListener(e-> calculateBill());
    }
    private void calculateBill() {
        int total=0;
        StringBuilder bill = new StringBuilder("Selected items: \n");
        if(pizza.isSelected()){
            total+=100;
            bill.append("Pizza -> 100rs \n");
        }
        if(burger.isSelected()){
            total+=50;
            bill.append("Burger -> 80rs \n");
        }
        if(juice.isSelected()){
            total+=50;
            bill.append("juice -> 50rs \n");
        }
        bill.append("Total: ").append(total).append("\n");
        output.setText(bill.toString());
    }
}
```

```
public static void main(String[] args) {  
    SwingUtilities.invokeLater()-> new FoodOrderApp();  
}  
}
```

Output:



Lab Assignment Task 2

StudentGradeEvaluator.java:

Code:

```
import javax.swing.*;
import java.awt.*;
public class StudentGradeEvaluator {
    public static void main(String[] args){
        JFrame frame = new JFrame("Grade Evaluator");
        frame.setLayout(new FlowLayout());
        JLabel front = new JLabel("Evaluate grade for Krish: ");
        JTextField marks1 = new JTextField(3);
        JTextField marks2 = new JTextField(3);
        JTextField marks3 = new JTextField(3);
        JButton evaluate = new JButton("Evaluate");
        JTextArea output = new JTextArea(5,50);
        evaluate.addActionListener(e->{
            try{
                int m1 = Integer.parseInt(marks1.getText());
                int m2 = Integer.parseInt(marks2.getText());
                int m3 = Integer.parseInt(marks3.getText());
                StringBuilder s = new StringBuilder();
                int total = m1+m2+m3;
                s.append("Total marks: ").append(total).append("\n");
                float average = (float)total/3;
                s.append("Average marks: ").append(average).append("\n");
                char grade;
                if(average>=75){
                    grade = 'A';
                }
                else if(average>=60){
                    grade = 'B';
                }
                else if(average>=40){
                    grade = 'C';
                }
                else{
                    grade = 'F';
                }
                s.append("Grade: ").append(grade);
                output.setText(s.toString());
            }
            catch(Exception ex){
                output.setText("Invalid input Marks");
            }
        });
        frame.add(front);
```



```

frame.add(marks1);
frame.add(marks2);
frame.add(marks3);
frame.add(evaluate);
frame.add(new JScrollPane(output));
frame.setSize(300,200);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
}

```

Output:

Grade Evaluator

Evaluate grade for Krish:

Total marks: 272
 Average marks: 90.666664
 Grade: A

Grade Evaluator

Evaluate grade for Krish:

Total marks: 207
 Average marks: 69.0
 Grade: B

Grade Evaluator

Evaluate grade for Krish:

Total marks: 175
 Average marks: 58.333332
 Grade: C

Grade Evaluator

Evaluate grade for Krish:

Total marks: 60
 Average marks: 20.0
 Grade: F

Post-Lab Questions

1. What is the role of JFrame?

Ans: JFrame is the main window of any swing application. It acts as a top level container that holds all other components like buttons, labels, and text fields. Without a JFrame, there is no visible window. It provides the title bar, borders and close/minimize/maximize controls. Every Swing app starts by creating a JFrame and adding componenets into it.

2. What is an event in GUI?

Ans: An event is any user action that the program needs to respond to. Examples include clicking a button, typing in a text field, moving the mouse or pressing a key. In Swing, events are automatically detected by the framework and passed to the appropriate listener if one has been registered. Events are what makes a GUI interactive rather than just static display.

3. How does ActionListener work?

Ans: ActionListener is an interface that listens for action events, most commonly button clicks. You attach it to a component using `addActionListener()`. When the user performs the action (like clicking the button), the `actionPerformed()` method is automatically triggered and executes whatever code you put inside it. In modern Java, this is written using lambda expression like `e → {..}` instead of writing a full class.

4. Why is exception handling required in GUI input?

Ans: Because users can type anything inside a text field – letters, symbols, spaces etc when numbers are expected. If we try to convert that input using `Integer.parseInt()` without a try-catch block and the user types “abc” the program will crash entirely. Exception handling catches that error gracefully and instead displays a helpful message like “Invalid Input”, keeping the application running smoothly.

5. What is the role of Timer in Swing?

Ans: `javax.swing.timer` is used to execute code repeatedly at fixed time intervals without freezing the GUI. For Example, it can be used to build a countdown clock, animate something on screen, or auto-refresh data every few seconds. It is important to use Swing’s own Timer (not `java.util.Timer`) because Swing is single-threaded – all UI updates must happen on Event Dispatch Thread (EDT) and Swings Timer runs on that thread automatically, keeping the interface responsive.